Find the Promos — Kuku AI Generalist, Problem 1

A rules-engine promo detector with a swappable LLM refiner, run on all 50 episodes of *I Married a Broken Soldier*. Author: vivek.

Why Problem 1, and the framing

I picked Problem 1 because it's the most bounded and checkable: a concrete input (50 screenplays), clear success criteria (20–60s clips that open on a hook and end on a cliffhanger, standalone), and a natural way to show careful AI use. The core question I framed it around: **how much do you give to AI vs. keep as code?** My answer — the model should never decide *where to cut*, only *how to say why*. So a deterministic, explainable rules engine finds and ranks every clip, and an optional LLM only rewrites the human copy. That keeps a bad model from ever corrupting a timestamp.

Assumptions I made where the brief was silent

(1) “Promo” = one continuous 20–60s stretch, so I find the single strongest stretch per moment and de-dup overlaps — most ReelShort/DramaBox clips work this way; multi-segment joins are a flagged next step. (2) “Standalone” is scored heuristically from dialogue share, speaker diversity and settings, since I can't watch footage. (3) Timestamps in the source are **M:SS.ss** (the seconds field carries its own decimal) and some headers are all-caps — both handled. (4) “Rank” = a 45/35/20 blend of hook / cliff / standalone; the top-N across the pack is the “post first” list.

How it works

Stage	What it does
parse	Every CAMERA/ACTION/SOUND/character beat -> a timeline; episodes = 50.
score	~14 weighted cue families (violence, reveal, power stakes, unresolved endings...) mark each beat.
search	All 20–60s windows whose edges snap to beats; score each on hook (first 3s), cliff (last beat), standalone.
dedup + rank	Drop clips sharing >40% of a stronger one; keep top-3 per episode; global rank.
copy	Deterministic hook/cliff/why lines (offline, zero cost). Optional LLM rewords copy only.
audit	Hard checks (duration, in-bounds, has dialogue) + soft flags (weak cliff, 3+ settings) -> ok/review/reject.

Running it (zero installs; Python 3.9+):

```
python -m kuku_promo_finder <episode-folder> -o output # core, stdlib only
python make_board.py output # self-contained board.html, no dependencies
streamlit run app.py # optional interactive board (pip install streamlit)
```

How it scales to every show we make

The pipeline is O(episodes x beats^2) and each episode is ≤ ~120 beats, so a full show runs in well under a second, no paid API, no GPU — it's a batch job you can point at a week's new shows. Cost at volume stays near zero; the only optional paid piece is copy-polish via an LLM, and I cap it (--llm-limit) and default it off. Because the model is a single swappable endpoint (any OpenAI-compatible API: Ollama local, OpenRouter free-tier, or a Kuku-provided API in the interview), swapping to a stronger model is one env var. To scale to new genres, the whole tuning surface is one lexicon (CUES) in scoring.py — raise reveal_secret for mysteries, physical_confrontation for action — without touching the search.

How I know it's good — and where it fails

I ran it on all 50 episodes and reviewed the picks by hand, then built the self-critique into the tool rather than the doc. On this pack: **144 promos, 125 clean / 19 flagged for review / 0 rejected.** Every clip is 20–60s and on-beat. The “post first” list reads like a real cut sheet — e.g. EP32 “Why?!” opening, EP47 “Juliana, she tried to destroy you!”, EP28 “I was poisoned!” — each cut lands on a genuinely unresolved beat. The 19 “review” flags are exactly the soft spots: 14 are “low cliff” (the tool cut on a calm/quiet moment, e.g. EP13 ends on “Gerald shouting down into the garden” — plausible but I'd want a human to confirm it hooks), and the rest are “spans 3 settings” or “low hook”.

Honest failure I caught and fixed

My first pass scored a “cliffhanger” as high whenever the episode simply ended. A calm fade-to-black looked identical to an ominous one, so I added a **resolution cap** (calm/exhale/leave endings clamp the cliff score) plus a question-mark/ellipsis cue, which removed the false cliffs. I also caught a parser bug where “Scene end” headers under-counted episode length (a trailing out-of-order SFX beat ran past them); I anchored episode start/end to the beats themselves, which took the reject count to 0. These are in the git history and the audit.

Where I used AI — and one place it got it wrong

I used Claude to scaffold the code and reason about the lexicon, and to generate this doc. One place it got it wrong: it proposed treating a beat’s End timestamp as “hours:minutes.tenths” (H:MM.F), reading 00:04.80 as ~4 minutes. It's actually M:SS.ss (04.80 seconds) — the seconds field has its own decimal. I verified against the logline + scene math and corrected the parser; the LLM had no way to know the format, so I anchored it to the data rather than its guess. I also deliberately kept the copywriter rule-based (Claude prose would over-sell clips); the model only refines copy if an endpoint is configured.

What's in the submission

Working prototype: this repo (kuku_promo_finder) with clear run steps + a dependency-free board.html and an optional Streamlit app. Raw outputs: output/ (promos_all.json, post_first, per-episode, failure_notes, quality_summary, digest.md). Docs: this 2-page PDF + walkthrough script. It runs on real inputs and produces output an editor can use: copy the timestamps straight to an NLE.